Lab 6
Process Control

TA: Reui-Lin Gao

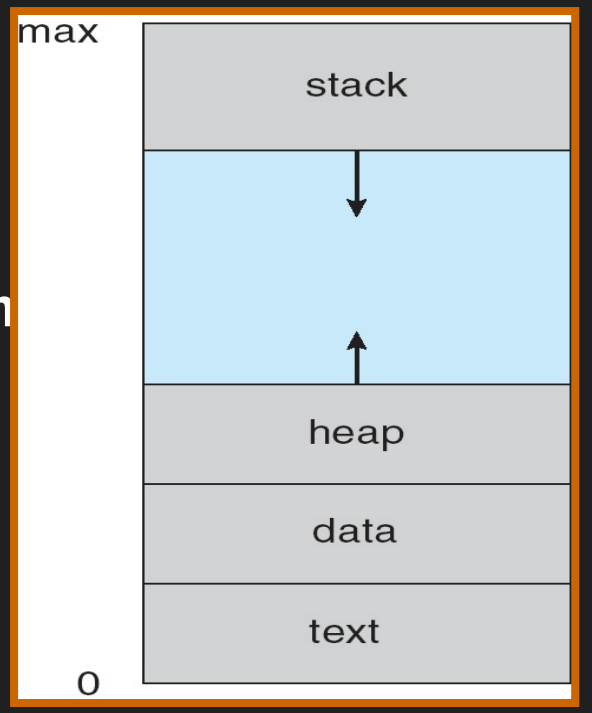
Prof.: Hsung-Pin Chang

NCHU System & Network Lab

$ Outline

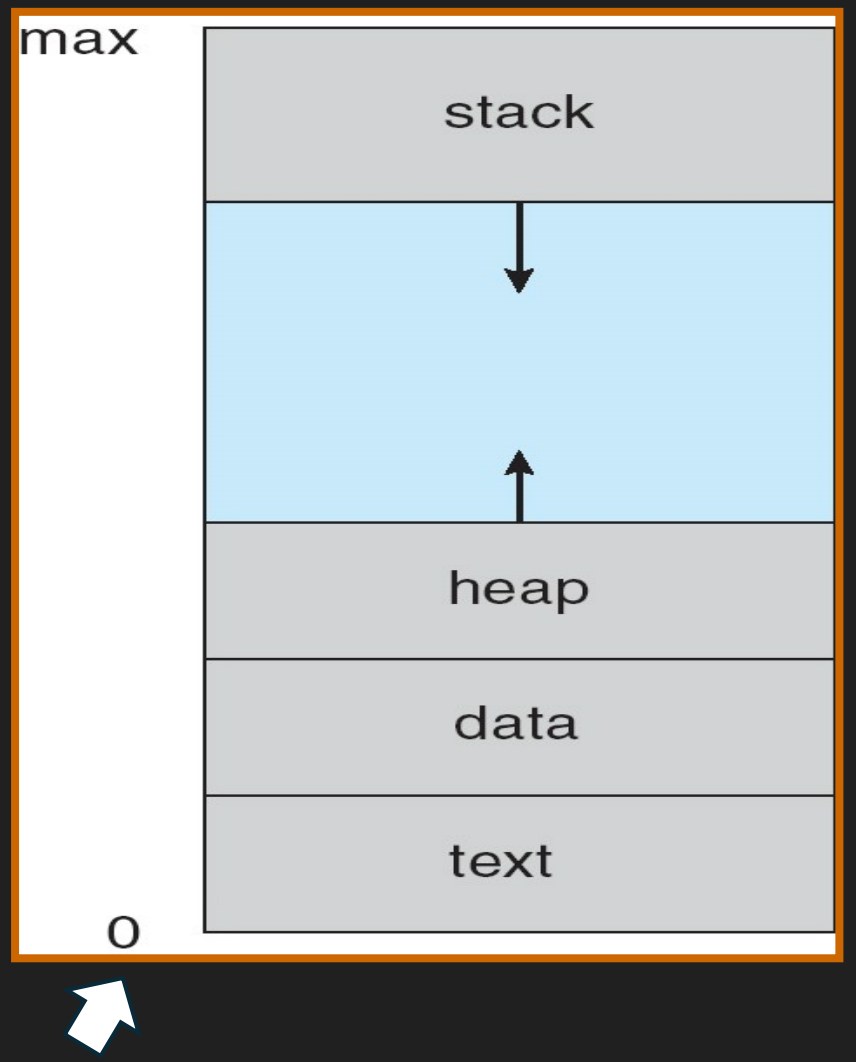
- Introduction
- Function
- Exercise

$ Introduction (1/4)



- What is a process?
  - Informally, a process is a program in execution
  - A process includes:
  - data section, which contains global variables
  - text section, which contains the program code
  - stack section, which contains temporary data
  - heap section, which contains dynamically allocated memory
  - A process is more than the program code, it also includes the current activity
  - Represented by the value of the program counter and the contents of the processor's registers.

$ Introduction (2/4)



temporary data

dynamically allocated memory

global variables

program code

Process base address

$ Introduction (3/4)

- Example in C

```c
#include<stdio.h>
#include<stdlib.h>

//global variable (data section)
int a = 10;

void function(){
    //local variable (stack section)
    int b = 20;
}
```

```c
int main(){
    //temporary data (stack section)
    int c = 30;

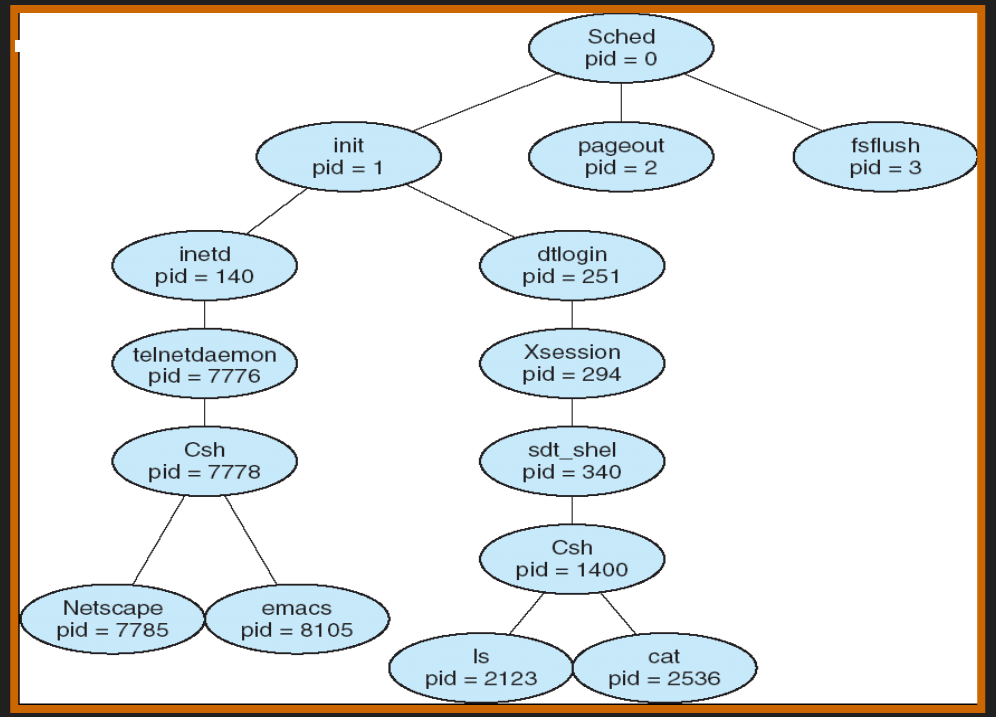
    //dynamic memory allocation (heap section)
    int *d = (int *)malloc(sizeof(int));
    *d = 40;

    function();
    free(d);

    return 0;
}
```

$ Introduction (4/4)

- Process Creation
  - Parent process create children processes, which, in turn create other processes, forming a tree of processes.



$ Outline

- Introduction
- Function
- Exercise

$ Function (1/13)

- The fork() Function:
  - fork() system call creates new process
  - fork() 會回傳新的 process 的 PID ，失敗則回傳 -1
  - fork() 會複製 parent’s memory image 給新的 child process 接收
  - 在 fork 之後，兩個 prcoess 都會繼續在各自的 memory images 執行

```
#include <unistd.h>

pid_t fork(void);
```

$ Function (2/13)

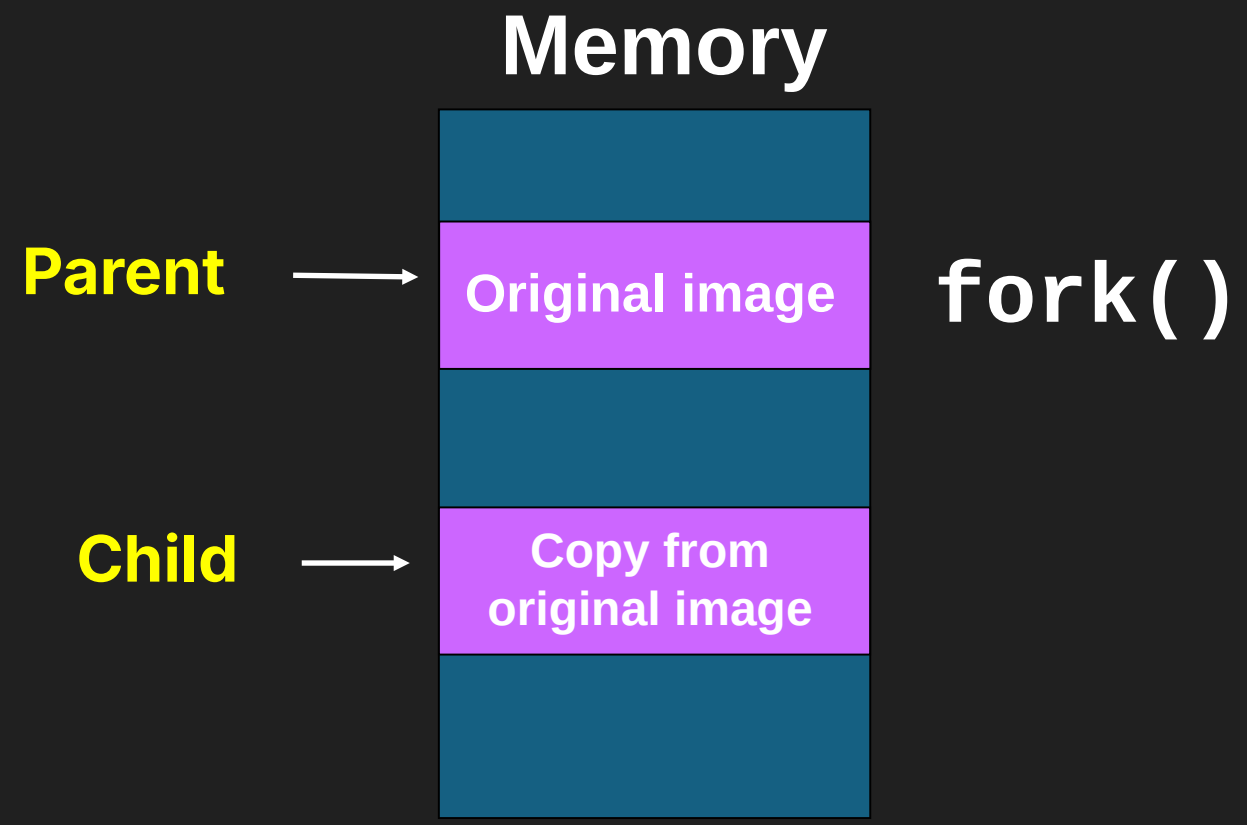
- Process Identification
  - UNIX identifies processes by a unique integral value called the process ID.
  - Each process also has a parent process ID, which is initially the process ID of the process that created it.

```
#include <unistd.h>

pid_t getpid(void); //return the process ID
pid_t getppid(void);     //return the parent
process ID
```

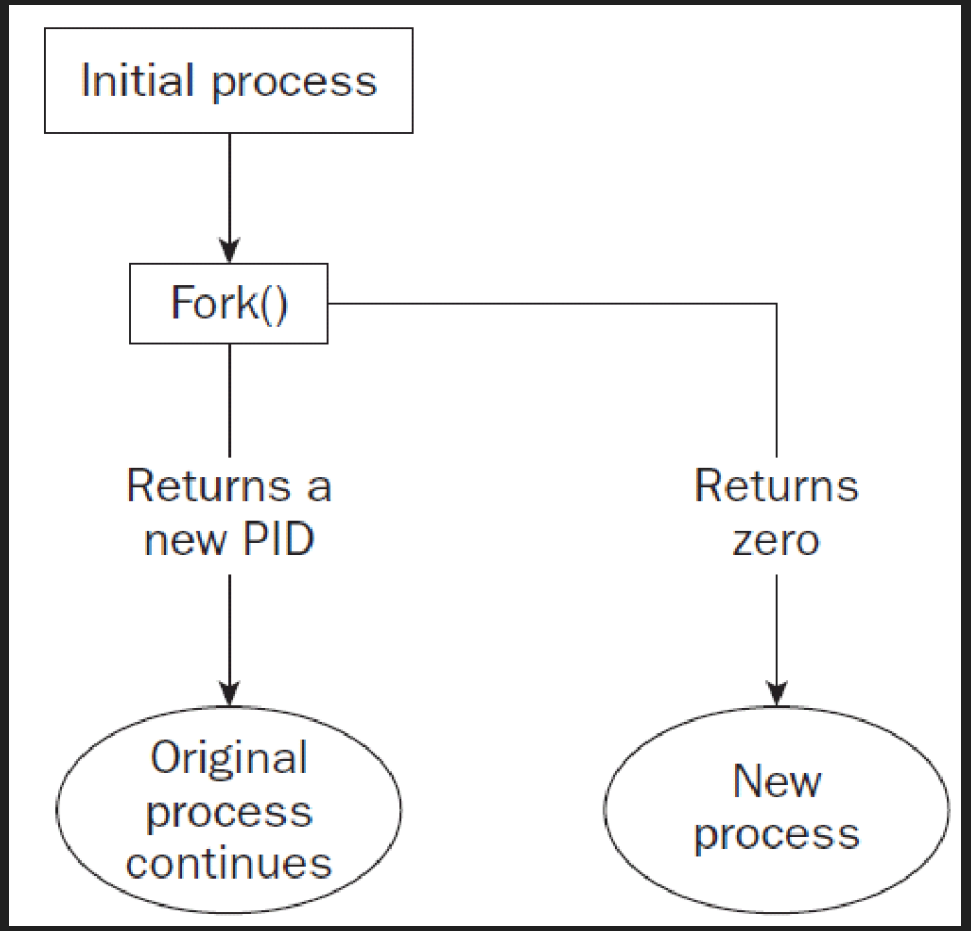
$ Function (3/13)

- The fork() Function



$ Function (4/13)

- The **fork()** Function



$ Function (5/13)

- Example (switch)

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./creat_process
------------this is child process------------
Pid : 5962
parent pid : 5961
-----------------------------------------
------------this is parent process-----------
Pid : 5961
child pid : 5962
-----------------------------------------
```

```c
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t new_pid;
    new_pid = fork();

    switch(new_pid){
        case -1:
            printf("fork error\n");
        case  0:
            printf("------------this is child process------------\n");
            printf("Pid : %d\nparent pid : %d\n", getpid(), getppid());
            printf("-----------------------------------------------\n");
            break;
        default:
            printf("------------this is parent process-----------\n");
            printf("Pid : %d\nchild pid : %d\n", getpid(), new_pid);
            printf("-----------------------------------------------\n");
            break;

    }
    return 0;
}
```

$ Function (6/13)

- Example (if)

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./creat_process_if
------------this is child process------------
Pid : 6069
Parent pid : 6068
-------------------------------------------
------------this is parent process-----------
Pid : 6068
Child pid : 6069
-------------------------------------------
```

```c
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t new_pid;
    new_pid = fork();

    if (new_pid < 0){
        printf("fork error\n");
        return 0;
    }
    else if (new_pid == 0){
        printf("-------------this is child process------------\n");
        printf("Pid : %d\nParent pid : %d\n", getpid(), getppid());
        printf("----------------------------------------------\n");
        return 0;
    }
    else{
        printf("-------------this is parent process-----------\n");
        printf("Pid : %d\nChild pid : %d\n", getpid(), new_pid);
        printf("----------------------------------------------\n");
        return 0;
    }
}
```

$ Function (7/13)

- The wait() Function
  - 當 parent 執行到 wait() ， parent 就會停止，直到他的 child 完成任務
  - 當 wait() 因為 child 的狀況回傳，會回傳該 child 的 PID
  - 如果 error 則會返回 -1

```
SYNOPSIS
#include <sys/wait.h>

pid_t wait(int *stat_loc);
```

$ Function (8/13)

- Example :

  fork() multiple times

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./fork_multi
I am child process 1.
Pid: 6087       Parent Pid: 6086
I am child process 2.
Pid: 6088       Parent Pid: 6086
I'm parent process.
Pid: 6086       Parent Pid: 5746
```

```c
#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>  //wait()

int main()
{
    if(fork() == 0)
    {
        printf("I am child process 1.\n");
        printf("Pid: %d\tParent Pid: %d\n", getpid(), getppid());
        return 0;
    }
    if(fork() == 0)
    {
        printf("I am child process 2.\n");
        printf("Pid: %d\tParent Pid: %d\n", getpid(), getppid());
        return 0;
    }
    wait(NULL);
    wait(NULL);
    printf("I'm parent process.\n");
    printf("PID: %d\tParent PID: %d\n", getpid(), getppid());
    return 0;
}
```

$ Function (9/13)

- Example

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./wait

------this is child process------
pid: 6169       parent pid: 6168

------this is parent process------
pid: 6168       child pid: 6169
```

Parent process always waits for child and executes last.

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t new_pid;
    new_pid = fork();

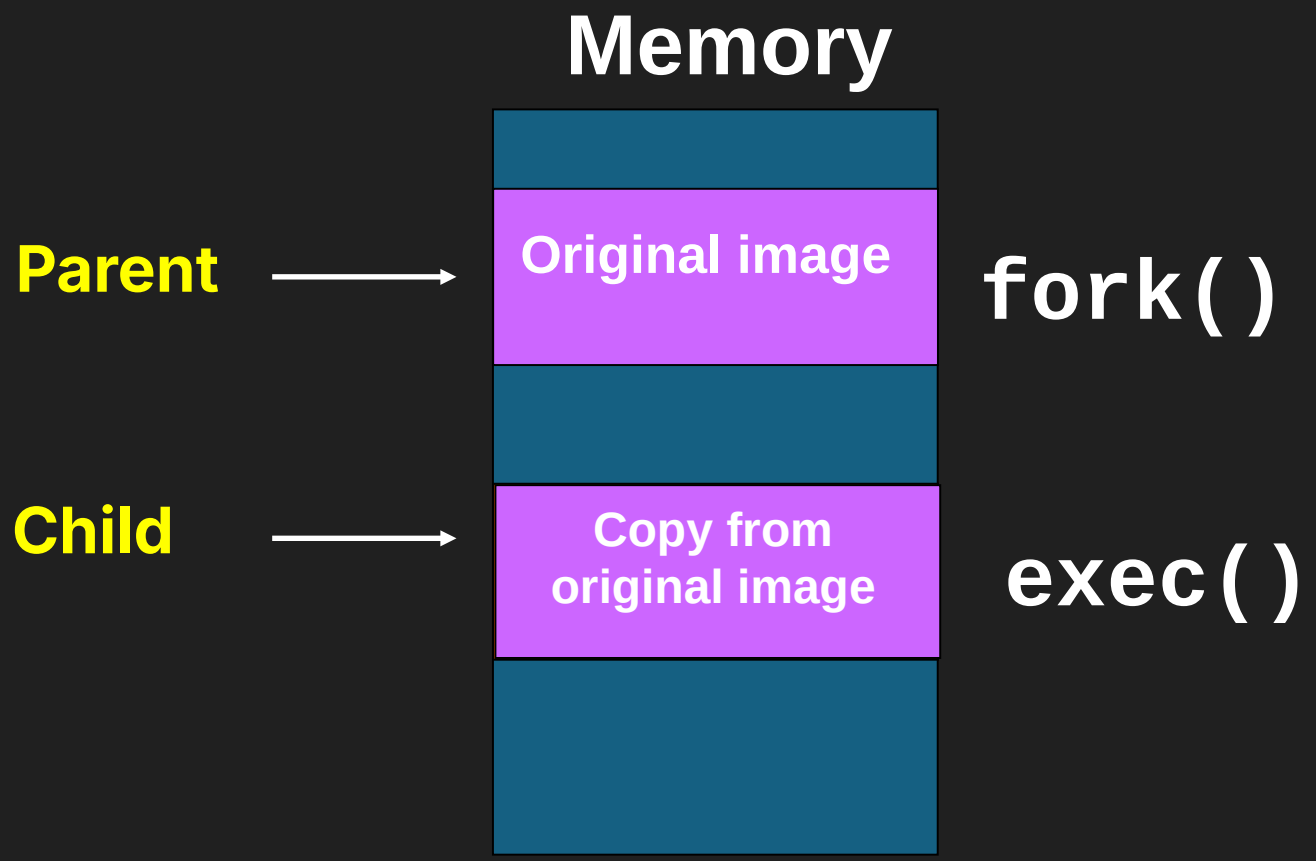
    switch (new_pid) {
        case -1:
            printf("fork error\n");
            break;
        case 0:
            printf("\n-------this is child process-------\n");
            printf("pid: %d\t parent pid: %d\n", getpid(), getppid());
            break;
        default:
            wait(NULL);
            printf("\n-------this is parent process------\n");
            printf("pid: %d\t child pid: %d\n", getpid(), new_pid);
            break;
    }
    return 0;
}
```

$ Function (10/13)

- The **exec()** Family
  - The **fork()** function creates a copy of the calling process, but many applications require the child process to execute code that is different from that of the parent.
  - The **exec()** family provides a facility for overlaying the process image of the calling process with a new image.
  - Use the **fork()–exec()** combination for the child to execute the new program while the parent continues to execute the original code.

$ Function (11/13)

- The exec() Family



$ Function (12/13)

- The exec() Family
  - execlp()
  - passes the command-line arguments in an explicit list and are useful if you know the number of command-line arguments at compile time.

```
#include <unistd.h>

int execlp (const char *filename, const char
*arg0, ... /*(char *)0 */);
```

$ Function (13/13)

- Example

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./exec
total 140
-rwxrwxr-x 1 wuyodi wuyodi 16136 Aug 26 20:13 creat_process
-rw-rw-r-- 1 wuyodi wuyodi   709 Aug 26 20:12 creat_process.c
-rwxrwxr-x 1 wuyodi wuyodi 16184 Aug 26 20:22 creat_process_if
-rw-rw-r-- 1 wuyodi wuyodi   707 Aug 26 20:22 creat_process_if.c
-rwxrwxr-x 1 wuyodi wuyodi 16288 Aug 26 20:11 ex1
-rw-rw-r-- 1 wuyodi wuyodi  1174 Aug 26 19:42 ex1.c
-rwxrwxr-x 1 wuyodi wuyodi 16168 Aug 26 20:11 ex2
-rw-rw-r-- 1 wuyodi wuyodi   777 Aug 26 19:41 ex2.c
-rwxrwxr-x 1 wuyodi wuyodi 16168 Aug 26 20:40 exec
-rw-rw-r-- 1 wuyodi wuyodi   741 Aug 26 20:40 exec.c
-rwxrwxr-x 1 wuyodi wuyodi 16176 Aug 26 20:27 fork_multi
-rw-rw-r-- 1 wuyodi wuyodi   606 Aug 26 20:27 fork_multi.c
-rwxrwxr-x 1 wuyodi wuyodi 16168 Aug 26 20:30 wait
-rw-rw-r-- 1 wuyodi wuyodi   694 Aug 26 20:30 wait.c

----- this is parent process ------
Pid : 6240       child pid : 6241
-----------------------------------
```

```c
#include<stdio.h>
#include<unistd.h>

int main()
{
    pid_t new_pid;
    new_pid = fork();

    switch(new_pid){
        case -1 :
            printf("fork error\n");
            break;
        case 0 :
            execlp("/bin/ls", "ls", "-l", NULL);
            break;
        default :
            printf("\n------this is parent process-------\n");
            printf("Pid : %d\t chile pid : %d\t", getpid(), new_pid);
            printf("\n-----------------------------------\n");
            break;
    }
    return 0;
}
```

$ Outline

- Introduction
- Function
- Exercise 1 、 2

$ Exercise 1 (40pts.) (1/6)

- Declare a **global variable** and a **local variable**
  - Note that, both the global and local variables are initialized to the same value (=10)
  - Print both the values of global and local variables in screen (With the process id and parent's id)

$ Exercise 1 (40pts.) (2/6)

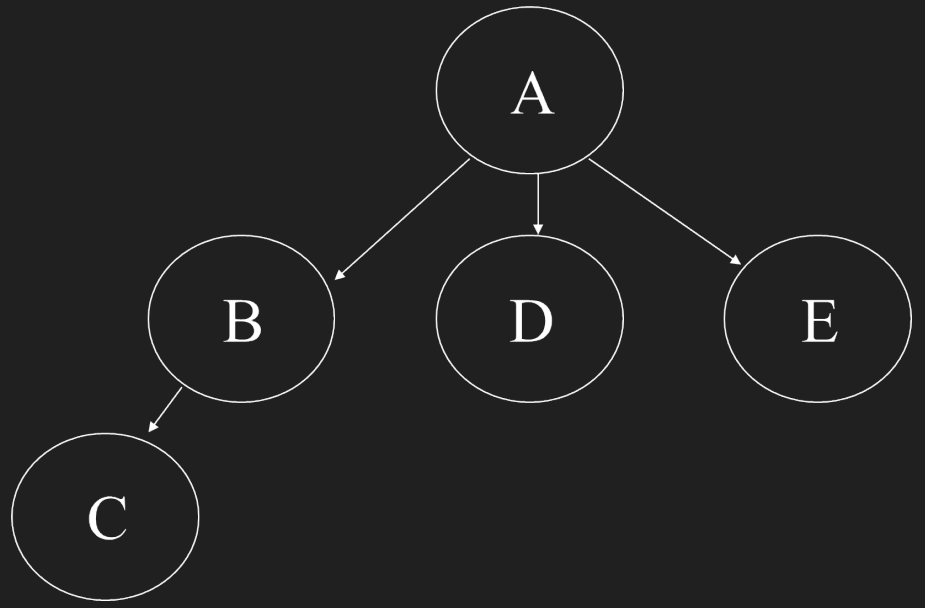
- **Create a child process**
  - Print both the values of global and local variables in screen
  - Then, increases both the values of the global variable and local variable
- Finally, both processes print the values of the global and local variables
  - With their process id and parent’s id.

$ Exercise 1 (40pts.) (3/6)

- **Result**

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./ex1
Initial value:
PID: 6461       ,PPID: 5746     , Global variable: 10   , Local variable: 10
---------------------------------------------------------------------------
Child process:
PID: 6462       ,PPID: 6461     , Global variable: 10   , Local variable: 10
Child process after increment:
PID: 6462       ,PPID: 6461     , Global variable: 11   , Local variable: 11
Parent process:
PID: 6461       ,PPID: 5746     , Global variable: 10   , Local variable: 10
Parent process after increment:
PID: 6461       ,PPID: 5746     , Global variable: 10   , Local variable: 10
---------------------------------------------------------------------------
```

$ Exercise 2 (50pts.) (4/6)



- Write a program that **creates 5 processes**, forming a tree configuration illustrated in the figure.
- Each process print the following report
  - Its' process id and the parent's process id

$ Exercise 2 (50pts.) (5/6)

- Each process, if it is a parent, must make a number of wait() system calls to wait until all of it's children exit.
  - The number of wait() system calls depend on the number of children

$ Exercise 2 (50pts.) (6/6)

- Result

```
wuyodi@wuyodi-VirtualBox:~/oslab/lab6$ ./ex2
Child process E -     PID: 6455     , PPID: 6454
Child process C -     PID: 6457     , PPID: 6456
Child process B -     PID: 6456     , PPID: 6454
Child process D -     PID: 6458     , PPID: 6454
Parent process A -     PID: 6454     , PPID: 5746
```

$ References

- Avi Silberschatz, Peter Baer Galvin and Greg Gagne, “Operating System Concepts,” John Wiley & Sons, 6th Edition, 2001
- “Unix Systems Programming: Communication, Concurrency, and Threads” by Kay A. Robbins, Steven Robbins
- Neil Matthew and Richard Stones, “Beginning Linux Programming,” Wiley publishing, 3rd Edition, 2004
- W. Richard Stevens, “Advanced Programming in the UNIX Environment,” Addison-Wesley, 1992



Thanks for your listening

Made by PurpleRed, Present on 2023/10/23